Individual Project 1: Professional Profile Website

WAPH - Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student: Artem Tikhonov

Email: tikhonam@mail.uc.edu

Course Repository: https://github.com/tartem26/waph-tikhonam.git

Individual Project 1 Folder: https://github.com/tartem26/tartem26.github.io

Deployed Website: https://tartem26.github.io/

WAPH Course Page: https://tartem26.github.io/waph.html

Figure 1: Artem's headshot

The Project's Overview

This individual project focused on building and deploying a professional profile website using front-end web development skills from the WAPH course. The website was deployed through GitHub Pages and is available publicly at `https://tartem26.github.io/`.

The main page, `index.html`, works as my professional profile and resume website. It includes my contact information, education, skills, work experience, relevant

course projects, awards, community activities, and hobbies. I also created a separate WAPH course page, `waph.html`, to introduce the course and link my course work, including labs, hackathons, and projects.

The technical part of the project includes an open-source CSS framework, JavaScript features from Lab 2, jQuery, another JavaScript library, two public APIs, JavaScript cookies, and a page tracker. I also included a disclaimer for the third-party API content because the jokes and images are generated by public external services.

Files Used

The main files and folders used for this individual project are:

- `index.html`
- `waph.html`
- `assets/css/style.css`
- `assets/js/main.js`
- `assets/img/headshot.jpg`

The website files and the project report are stored in the GitHub Pages repository:

`https://github.com/tartem26/tartem26.github.io`

No screenshots are included in this report because the project is deployed and publicly visible online. The deployed website and WAPH course page are linked above.

General Requirements

The project required a professional website deployed on GitHub Pages. I implemented the website using a public `github.io` repository, so the site can be viewed directly in a browser without running a local server.

The professional profile page includes my resume content, including my name, title, location, email, phone number, LinkedIn, GitHub, skills, work experience, education, course projects, awards, community activities, and hobbies.

The separate WAPH page introduces the Web Application Programming and Hacking course. It also links to my labs, hackathon work, and Individual Project 1 folder in my course repository.

Non-Technical Requirements

Professional Profile and Resume

The homepage was designed as a professional resume-style website. I used my CV content to build the page sections.

The professional profile includes the following sections:

- Skills
- Work Experience
- Education
- Relevant Course Projects
- Awards
- Community Activities
- Hobbies

The relevant course project titles are linked to their GitHub repositories:

- Instrument Counter
- Campus Navigation
- Bankruptcy Risk
- Custom BBRv2
- AI Exercise Assistant

WAPH Course Page

I created a separate page named `waph.html` for the WAPH course. This page introduces the course and summarizes the coursework. It includes links to the course repository and the folders for Lab 0, Lab 1, Lab 2, Hackathon 1, and Individual Project 1.

The WAPH page is available here:

`https://tartem26.github.io/waph.html`

Open-Source CSS Framework

I used Bootstrap as the open-source CSS framework for the website layout, navigation bar, responsive grid, buttons, and general structure.

The Bootstrap CSS file is loaded in both `index.html` and `waph.html`:

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="st
```

I also wrote a custom CSS file to make the website more professional and personalized:

`assets/css/style.css`

The custom CSS adds the hero section, dark professional layout, cards, timeline style, responsive spacing, project cards, and interactive section styling.

Page Tracker

I added a page tracker to the homepage inside the interactive requirements section. The tracker is displayed as a visit badge and automatically counts page visits when the deployed website is loaded.

The tracker is included in `index.html` using an image badge:

```
<a href="https://hits.seeyoufarm.com" target="_blank" rel="noopener">
  <img src="https://hits.seeyoufarm.com/api/count/incr/badge.svg?url=https%3A%2F%2Ftartem26.
</a>
```

Technical Requirements

JavaScript Libraries

The project required jQuery and one more open-source JavaScript framework or library. I used:

- jQuery
- Day.js

jQuery is used for DOM manipulation, click events, and API requests. Day.js is used to format dates and times for the digital clock, footer year, and cookie visit message.

The libraries are loaded in both pages:

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/dayjs@1/dayjs.min.js"></script>
```

Digital Clock

I implemented a digital clock in `assets/js/main.js`. The clock updates every second and displays the current day, date, and time.

The JavaScript function is:

```
function updateDigitalClock() {
  const currentTime = dayjs().format("dddd, MMMM D, YYYY h:mm:ss A");
  $("#digitalClock").text(currentTime);
}
```

The function is started when the page loads:

```
updateDigitalClock();
setInterval(updateDigitalClock, 1000);
```

Analog Clock

I implemented an analog clock using the HTML5 `<canvas>` element. The clock is drawn with JavaScript and updates every second.

The HTML element is:

```
<canvas id="analogClock" width="180" height="180" aria-label="Analog clock"></canvas>
```

The JavaScript uses the canvas context to draw the clock face, numbers, and hands. The clock is refreshed every second:

```
drawAnalogClock();
setInterval(drawAnalogClock, 1000);
```

Show / Hide Email

I implemented a show/hide email button using jQuery. The email is hidden by default. When the user clicks the button, the email appears as a clickable mail link. Clicking the button again hides it.

The button is in `index.html`:

```
<button id="emailToggle" class="btn btn-primary">Show my email</button>
<p id="emailArea" class="mt-3"></p>
```

The JavaScript code changes the button text and updates the email area:

```
function setupEmailToggle() {
  let emailVisible = false;

  $("#emailToggle").on("click", function () {
    if (emailVisible) {
      $("#emailArea").empty();
      $("#emailToggle").text("Show my email");
      emailVisible = false;
    } else {
      const emailLink = $("<a>")
        .attr("href", "mailto:tikhonam@mail.uc.edu")
        .text("tikhonam@mail.uc.edu");
      $("#emailArea").empty().append(emailLink);
      $("#emailToggle").text("Hide my email");
      emailVisible = true;
    }
```

```
  });
}
```

Extra JavaScript Functionality

For the additional JavaScript functionality of my choice, I implemented a dark/light theme toggle. The button appears in the navigation bar and lets the user switch the website theme.

The button is:

```
<button id="themeToggle" class="btn btn-sm btn-outline-light">Toggle Theme</button>
```

The selected theme is saved using `localStorage`, so the page remembers the user's selected theme after refresh.

```
function toggleTheme() {
  document.body.classList.toggle("dark-mode");
  const theme = document.body.classList.contains("dark-mode") ? "dark" : "light";
  localStorage.setItem("siteTheme", theme);
}
```

This was my chosen extra functionality because it improves the user experience and makes the website feel more interactive.

Web API Integration

JokeAPI

The first public API is JokeAPI. The project required the JokeAPI with the `Any` category and a new joke every 1 minute. I used this endpoint:

```
https://v2.jokeapi.dev/joke/Any?safe-mode&type=single
```

The JavaScript function requests a joke and displays it on the page:

```
function loadJoke() {
  const jokeUrl = "https://v2.jokeapi.dev/joke/Any?safe-mode&type=single";

  $.getJSON(jokeUrl)
    .done(function (data) {
      if (data && data.joke) {
        $("#jokeText").text(data.joke);
      } else {
        $("#jokeText").text("No joke was returned by the API.");
      }
    })
```

```
    .fail(function () {
      $("#jokeText").text("The JokeAPI request failed. Please try again later.");
    });
}
```

The joke is loaded when the page opens and then refreshes every 1 minute:

```
loadJoke();
setInterval(loadJoke, 60000);
```

I also added a button so the user can manually load a new joke.

Dog CEO Image API

The second public API is Dog CEO API, which provides random dog images. I used it as the required public API with graphics.

The endpoint is:

```
https://dog.ceo/api/breeds/image/random
```

The JavaScript function requests a random image and displays it on the page:

```
function loadDogImage() {
  const dogUrl = "https://dog.ceo/api/breeds/image/random";

  $.getJSON(dogUrl)
    .done(function (data) {
      if (data && data.message) {
        $("#dogImage")
          .attr("src", data.message)
          .attr("alt", "Random dog from Dog CEO API");
      }
    })
    .fail(function () {
      $("#dogImage")
        .attr("alt", "The Dog CEO API request failed. Please try again later.");
    });
}
```

I also added a button so the user can manually load a new image.

API Disclaimer

I included a disclaimer in the API section because the content comes from third-party public services:

```
Disclaimer: The jokes and dog images shown in this section are generated by public third-pa
```

This disclaimer is visible on the deployed website.

JavaScript Cookies

The project required JavaScript cookies to remember the client. I implemented a cookie named `lastVisit`.

If the user visits the website for the first time, the page displays:

`Welcome to my homepage for the first time!`

If the user revisits the website, the page displays:

`Welcome back! Your last visit was <the date/time of last visit>`

The main JavaScript functions are:

```
function setCookie(name, value, days) {
  const expires = new Date(Date.now() + days * 24 * 60 * 60 * 1000).toUTCString();
  document.cookie = `${name}=${encodeURIComponent(value)}; expires=${expires}; path=/; SameS
}

function getCookie(name) {
  const cookies = document.cookie.split("; ");
  for (const cookie of cookies) {
    const [cookieName, cookieValue] = cookie.split("=");
    if (cookieName === name) {
      return decodeURIComponent(cookieValue);
    }
  }
  return "";
}

function updateVisitMessage() {
  const lastVisit = getCookie("lastVisit");
  const now = dayjs().format("MMMM D, YYYY h:mm:ss A");

  if (!lastVisit) {
    $("#visitMessage").text("Welcome to my homepage for the first time!");
  } else {
    $("#visitMessage").text(`Welcome back! Your last visit was ${lastVisit}`);
  }

  setCookie("lastVisit", now, 365);
}
```

Deployment

The website was deployed through GitHub Pages using the repository:

`https://github.com/tartem26/tartem26.github.io`

The main professional profile website is available at:

`https://tartem26.github.io/`

The WAPH course page is available at:

`https://tartem26.github.io/waph.html`

The files were committed and pushed to GitHub after testing locally.

Testing and Observations

I tested the homepage and WAPH page in the browser after deployment. The navigation links work correctly, and the professional profile content is visible on the homepage. The WAPH page opens separately and links to my course work.

The digital clock and analog clock update every second. The show/hide email button displays and hides the school email correctly. The theme toggle changes the page theme and remembers the selected theme after refresh.

The JokeAPI section loads a joke from the `Any` category when the page opens, and it refreshes every 1 minute. The Dog CEO API section loads a random image and allows the user to request another image. The cookie message changes depending on whether the user is visiting the page for the first time or returning later.

The page tracker is visible in the interactive section and counts visits automatically. Since the project is deployed online, the main evidence for the project is the live GitHub Pages website.

Report Generation

This report was written in Markdown in the following file:

`https://github.com/tartem26/tartem26.github.io/blob/main/README.md`

After finishing the website and checking the deployed pages, I generated the PDF report using `pandoc`.

The command I used was:

```
cd ~/waph-tikhonam/individual-project1
pandoc README.md -o tartem26-waph-individual-project1.pdf
```

The final PDF file for submission is:

`tartem26-waph-individual-project1.pdf`

Conclusion

This individual project helped me connect the front-end web development skills from Lab 2 with cloud deployment through GitHub Pages. I built a professional profile website, created a separate WAPH course page, and added interactive JavaScript features such as clocks, email hiding, cookies, and a theme toggle.

The project also gave me more practice with Ajax-style API integration. I used public APIs to load dynamic content into the page and handled the returned JSON data in JavaScript. The biggest takeaway for me is that a static website can still feel interactive and dynamic when JavaScript, APIs, cookies, and cloud deployment are combined correctly.